

Development of a RL Agent to Drive in CarRacing-v2



Reinforcement Learning
Oral Presentation

CSC_52081_EP

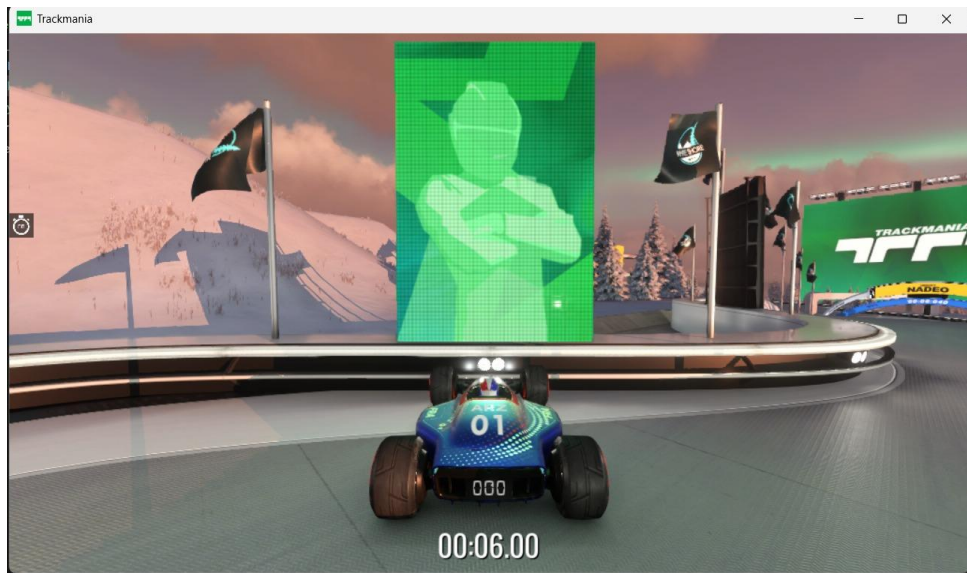
Timothée BESSARD - Tom LEON

Cedric TRINH - Henri-Pierre GAWEL - Titouan SALIN

Learning how to drive with TrackMania ?

TrackMania (initial environment)

- **Runs in real time** → very slow data collection
- Single game instance → **no parallel training** on one PC
- ~30s per episode → data collection becomes the bottleneck
- After several hours, the agent still could not stay on track

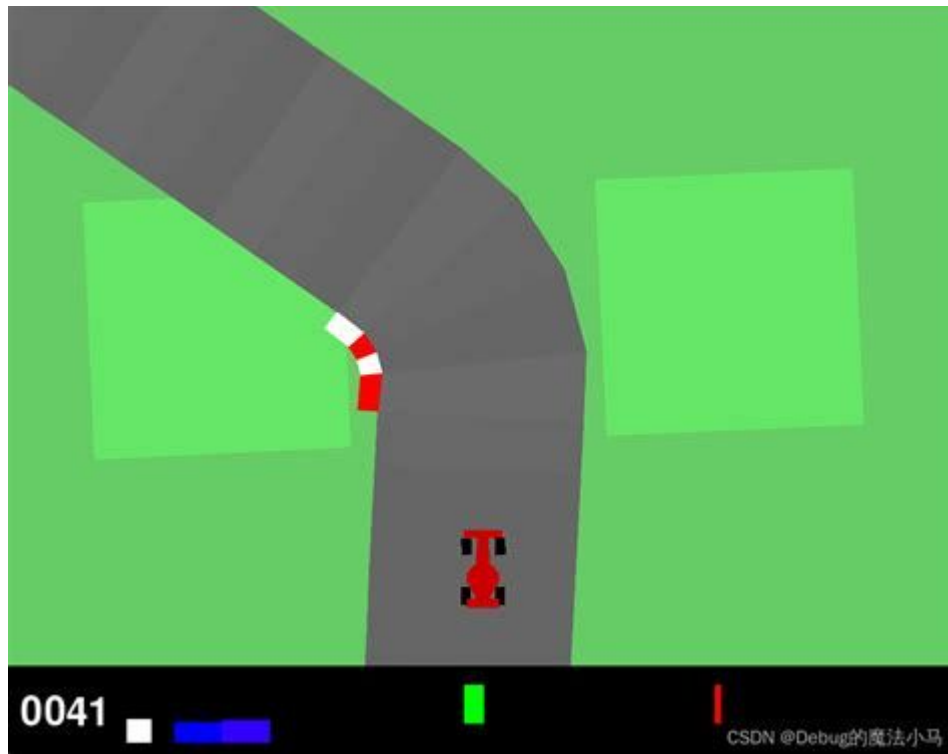


RacingCar : Lightweight simulation + Running faster than real time

→ Allows much faster episode generation

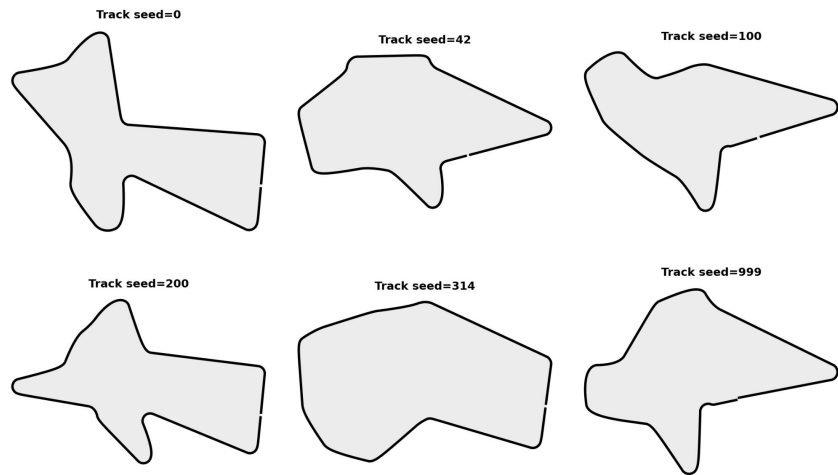
CarRacing as a Continuous Control Problem

- Vision-based observations
- Continuous actions : steering, gas, brake
- Nonlinear dynamics
→ harder than simple grid or discrete environments.
- Standard benchmark for Deep RL



Experimental protocol

Randomly Generated CarRacing-v3 Tracks



- 100,000 training timesteps
- Training seeds: 0, 1, 2
- Training tracks: randomly generated at each episode
- The mean reward is computed over 5 random circuits.

A first algorithm : Proximal Policy Optimisation

PPO theory reminder

Motivation : Policy Gradient updates can be unstable (large updates $\rightarrow$ collapse)

Goal : improve the policy while staying close to the previous one

$\Rightarrow$ Use a clipped objective function :

$$J^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\omega_t(\theta) \hat{A}(s_t, a_t), \underbrace{\text{clip}(\omega_t(\theta), 1 - \epsilon, 1 + \epsilon)}_{\omega_t(\theta)\text{-clipped}} \hat{A}(s_t, a_t) \right) \right], \quad \omega_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

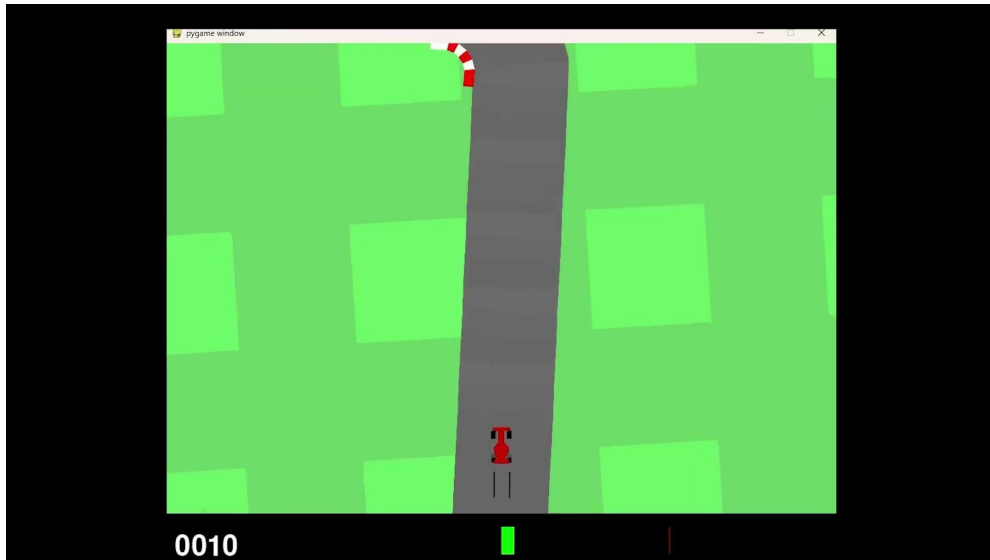
$\omega_t(\theta)$ measures how much the policy changed – if change too large $\rightarrow$ clip it

*prevents destructive
updates*

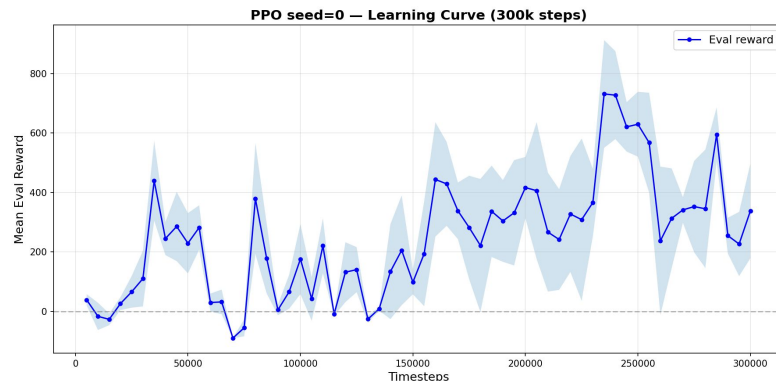
Result : $\left\{ \begin{array}{l} \bullet \text{ stable learning} \\ \bullet \text{ simple implementation} \end{array} \right.$

A first algorithm : Proximal Policy Optimisation

PPO results



Mean reward: 759

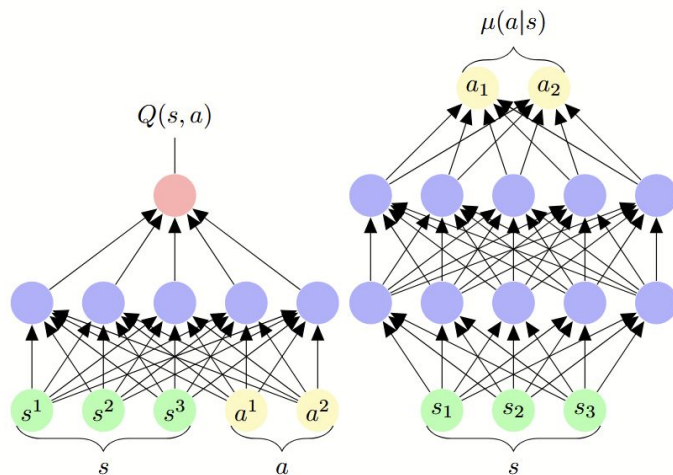


→ On-policy method => requires many interactions with the environment.

Deep Deterministic Policy Gradient (DDPG)

DDPG theory

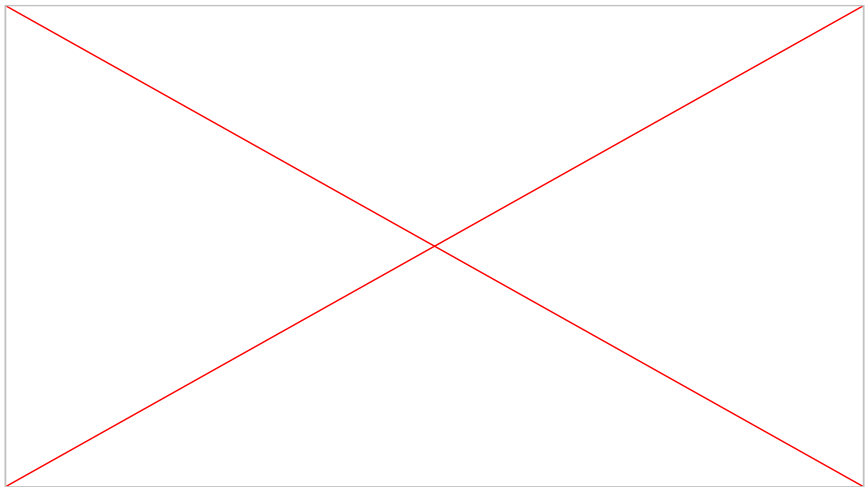
- Objective: extend DQN to continuous actions
- DDPG = DQN + DPG
- Actor-Critic architecture
- Loss function: $J(\theta) = \mathbb{E}_{s \sim \mathcal{B}} [Q_{\phi}(s, \mu_{\theta}(s))]$
- Exploration noise: $a = \mu_{\theta}(s) + \epsilon$



DDPG (Critic and Actor networks)

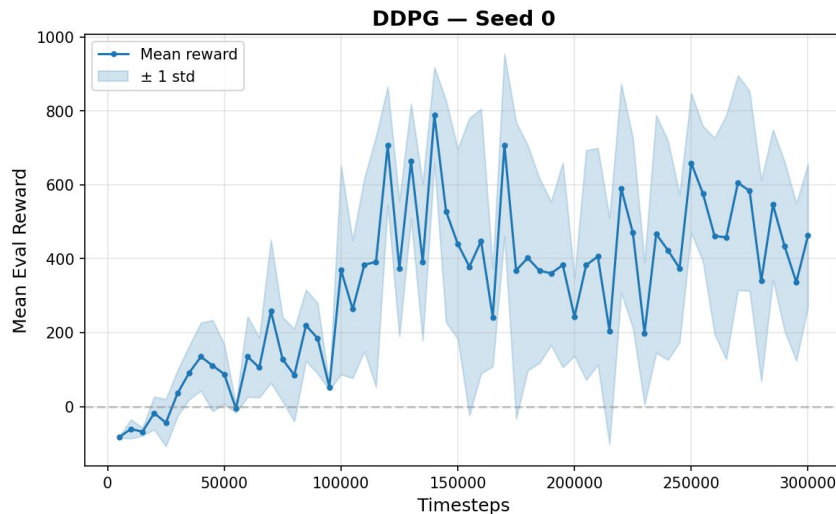
Deep Deterministic Policy Gradient (DDPG)

DDPG results



Mean reward: 687

Seed dependent



Deterministic policy → noise-based exploration → hyperparameter sensitivity ⇒ SAC

Soft Actor Critic (SAC)

SAC theory

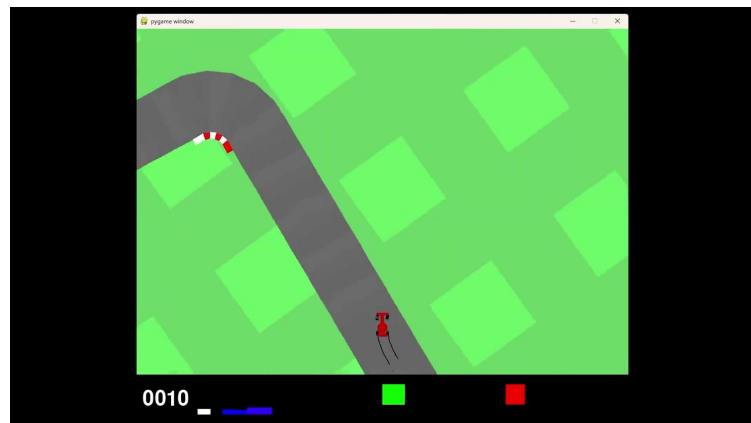
Stochastic policy $a \sim \pi(\cdot|s)$

2 critics

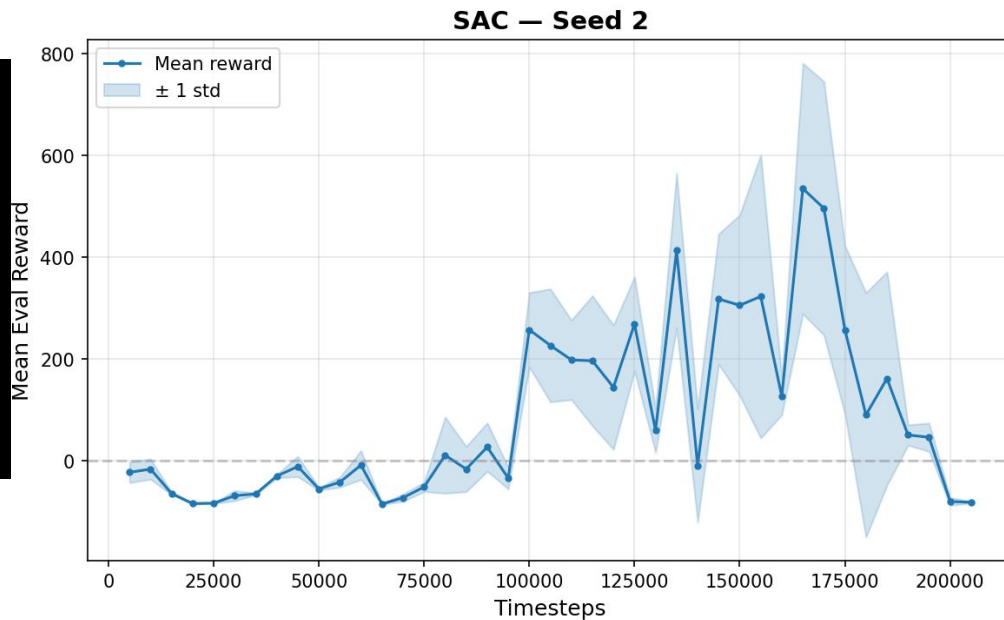
Maximize $\mathbb{E} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$

Soft Actor Critic (SAC)

SAC results



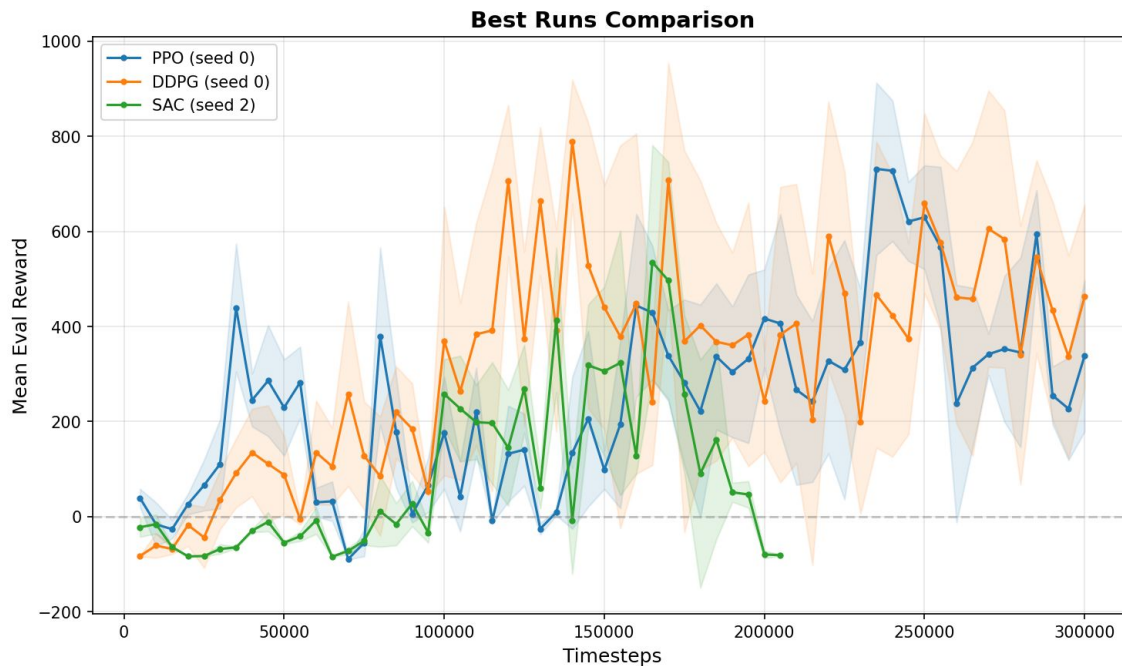
Mean reward : 611.1



Computationally heavy - Hyperparameters tuning difficult - Slower sometimes

Performance Comparison between PPO, DDPG and SAC

Learning Speed

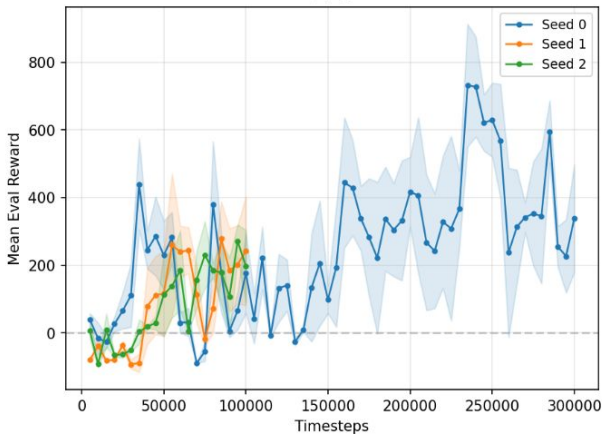


Performance Comparison between PPO, DDPG and SAC

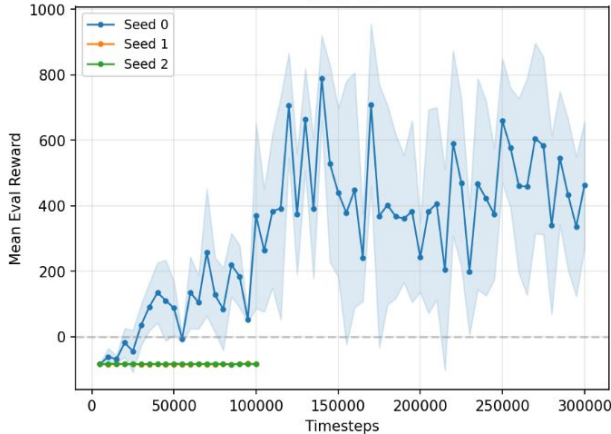
Stability

Learning Curves per Seed

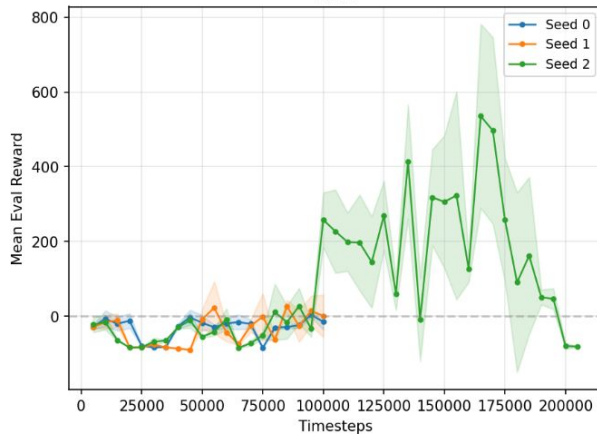
PPO



DDPG

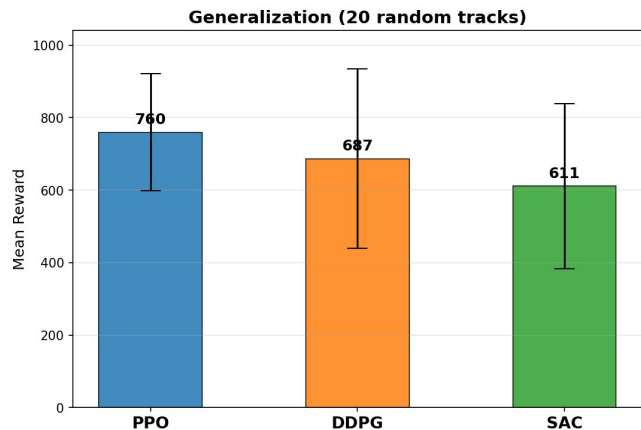


SAC



Conclusion

Performances



Algo	Mean	Std	Min	Max
PPO	759.9	161.2	300.6	905.2
DDPG	687.0	247.9	3.3	892.1
SAC	611.1	228.5	89.2	874.8

PPO → stability & robustness -> best short-budget performance

DDPG → off-policy efficiency -> but fragile

SAC → Advanced exploration -> needs more training